

Track Storefront Next Activity with Analytics

Monitor your storefront activity by collecting analytics data. Storefront Next provides a built-in solution for analytics, which is integrated with Einstein and Active Data. Implement analytics by using the analytics hook and events. Also, you can create integrations with other analytics systems.

How It Works

Storefront Next pages invoke the analytics hook to trigger the creation of `AnalyticsEvent` objects.

For example, on the product details page, you see this script.

```
    
1  const analytics = useAnalytics(  
2  
3  ...  
4  useEffect(() => {  
5    ...  
6    void analytics.trackViewProdu  
7      product: productData,  
8    });  
9    ...  
10 }
```

The `trackViewProduct` method creates a `ViewProduct AnalyticsEvent` that's passed into an internal `track` function. This `track` function sends the `AnalyticsEvent` to registered `EngagementAdapters` that handle the process of sending the event through to a target analytics system.

Contents

How It Works

[Set Up Einstein Activities](#)[Set Up Active Data](#)[Analytics Events](#)[Engagement Adapters](#)[See Also](#)

Set Up Einstein Activities



[Activities](#) in the *B2C Commerce Einstein API Guide*.

2. In `config.server.ts`, update the Einstein configuration. In the `einsteinId` field, specify the ID of the API client you created in the Einstein Configurator.

```
1 engagement: {
2     adapters: {
3         einstein:
4             enabl
5             host:
6             einst
```

Set Up Active Data

To receive Active Data events, set up Active Data in Business Manager. Active Data in Storefront Next supports product views on the Product Detail Page (PDP) and product impressions on a Product List Page (PLP) or recommendation.

Prerequisites

- Configure Active Data in Business Manager by following the steps in the next sections.

Step 1: Enable Data Collection

Open a support case asking Salesforce Customer Support to enable data collection.

Step 2: Confirm That Sites Are Online

1. Log into Business Manager.
2. Click App Launcher  and then select **Administration > Sites > Manage Sites**.
3. Confirm that the status is **Online** for the sites for which you want to track metrics. Active Data is collected only for live (online) sites.



Enabled

1. Log into Business Manager.
2. Click App Launcher  and then select **Merchant Tools > Site Preferences > Privacy**.
3. Confirm that Tracking (Default for New Storefront Session) is set to **Enabled**. Active data events are sent from Storefront Next to the `__Analytics-Start` controller in B2C Commerce. One way you can confirm that active data tracking is working is by viewing network requests and checking if the request to `__Analytics-Start` is responding with an `HTTP 200` code.

Step 4: Configure Active Data

1. In `config.server.ts`, enter the Active Data host. In the `host` field, specify the URL of the B2C Commerce instance.



```
1 activeData: {
2
3
4
5
6
7
```

2. If you have a hybrid storefront, specify the site UUID in the `siteUUID` field. In SFRA, get the site UUID from the value that comes after `dwac_` in the cookie in SFRA. This cookie appears only when a user is logged into SFRA.



Note

If `siteUUID` isn't set for the hybrid storefront, all Active Data events are treated as having come from anonymous guest users. In pure Storefront Next and PWA Kit Composable implementations, there is no `dwac_` cookie, so all



Analytics Events

Storefront Next provides these built-in event types.

- `ViewPageEvent`
- `ViewProductEvent`
- `ViewSearchEvent`
- `ViewCategoryEvent`
- `ViewRecommenderEvent`
- `ClickProductInCategoryEvent`
- `ClickProductInSearchEvent`
- `ClickProductInRecommenderEvent`
- `CartItemAddEvent`
- `CheckoutStartEvent`
- `CheckoutStepEvent`

Each event type defines different fields corresponding to the data required by that specific event.

All types extend `BaseEvent`, which provides the customer fields that are shared across all events, such as `usid` or `customerId`. All events also define a string `eventType` field that EngagementAdapters use during event handling. All types are included as part of the union type `AnalyticsEvent`.

For example, a `ViewProductEvent` Analytics event triggers both an Einstein `ViewEvent` and an Active Data product view with the default setup. The `ViewSearchEvent` and `ViewCategoryEvent` Analytics events similarly send the `ViewSearch` and `ViewCategory` Einstein events alongside Active Data product impressions.

Adding More Event Types

Storefront Next uses typescript module augmentation to extend existing events and add new event types. To add new event types, use the `AnalyticsEventExtensions` interface. Types added to `AnalyticsEventExtensions` are included in the `AnalyticsEvent` union type.



1. In a TypeScript declaration file, define our new event type. The new event must extend `BaseEvent` and define the `eventType` field. We also include a `cartItems` field for the items we're removing from cart.

```
1 declare module "@salesforce/sto
2   interface AnalyticsEventExten
3     CartItemRemove: BaseEvent &
4       eventType: "cart_item_rem
5       cartItems: Array<BasketPr
6     };
7   }
8 }
```

2. Update the analytics hook to support the new event.

```
1 /**
2  * Track removing items from ca
3  */
4 const trackCartItemRemove = asy
5   cartItems: ShopperBasketsV2.s
6 }) => {
7   return trackEvent(authPromise
8     cartItems: data.cartItems,
9   });
10 };
```

The `trackEvent` function creates the `CartItemRemove` event from the event type, handles Do Not Track, and adds session information such as the `usid` to the event.

3. Update engagement adapters to handle the new event type.

The `track` function sends `CartItemRemove` events to all registered engagement adapters. Therefore, the adapters need to know if they must handle or ignore `CartItemRemove` events.

4. Configure Engagement adapters to handle or ignore an event in `config.server.ts`. To have the Einstein adapter process `CartItemRemove` while having the Active Data adapter ignore the event, make the following changes inside `config.server.ts`.





```
6         // ... existing
7         cart_item_remov
8     },
9 },
10   activeData: {
11     // ... other config
12     eventToggles: {
13       // ... existing
14       cart_item_remov
15     },
16   },
17 },
18 }
```

5. Next, update the existing adapters that process `CartItemRemove`. Each adapter handles the event differently. For example, if we want to send `CartItemRemove` to a non-existent Einstein activity called `removeFromCart`, make this change.

```
</> | 📄
1 // Maps event types to Einstein
2 const einsteinEventToEndpointMa
3   // ... other event mappings
4   cart_item_remove: 'removeFr
5 };
6
7 // Converts events into a paylo
8 function convertEventToEinstein
9   // ... other events
10  case 'cart_item_remove':
11    return {
12      ...baseActivity,
13      products: event.cartI
14        extractEinsteinIt
15    },
16  };
17
```

Note

Einstein doesn't have a `removeFromCart` endpoint. If this event is sent from your browser, you see Einstein return an HTTP 400 response.

6. Update the pages that fire the new event.

Now, our `CartItemRemove` event is ready to be used. The next steps are to update the components that handle cart item removal to invoke the analytics hook and test.

```
</> |
```



Engagement Adapters

Engagement Adapters define how an event are sent to an analytics system. Storefront Next includes adapters for Einstein Activities and Active Data.

Engagement adapters follow the `EngagementAdapter` interface:

```
1 export interface EngagementAdapter {
2   name: string;
3   sendEvent?: (event: AnalyticsEvent) => void;
4   send?: (url: string, options?: object) => void;
5 }
```

The key method used by the analytics system is the `sendEvent` function. The internal `track` function invokes this method when an event is received. The `sendEvent` function takes an `AnalyticsEvent` and converts that event into a payload that can be sent to the target analytics system.

As a best practice, Storefront Next uses `navigator.sendBeacon` to send analytics events to target systems because it's non-blocking.

Adding More Adapters

Keep the following in mind when implementing your own `EngagementAdapter`.

- Use `eventToggles` to determine which events the adapter processes. This helps fine-tune the adapter to only handle the events the target analytics system supports.
- `sendEvent` has two main tasks. Consider implementing these tasks as separate functions that `sendEvent` invokes. These tasks are:
 - Converting an `AnalyticsEvent` object into the appropriate payload for the target analytics system
 - Sending the payload to the target system



src/adapters.index.ts file and invoke the `addAdapter` method.

```
1 export function initialize
2   // ... other adapters
3
4   addAdapter(
5     'custom-adapter', {
6       name: 'custom-adapte
7       sendEvent: async (
8     }
9   )
```

As a best practice, adapters provide a function for creating the adapter so you're more likely to see this script.

```
1 import {createCustomAdapter} fr
2
3 export function initializeEngag
4
5   // ... other adapters
6
7   addAdapter(
8     'custom-adapter',
9     createCustomAdapter()
10  )
```

See Also

[Einstein Activities Active Merchandising](#)

**DID THIS
ARTICLE SOLVE
YOUR ISSUE?**

Let us know so
we can improve!

[Share your feedback](#)